PAPER_189: S-C Scientific Calculator Architecture — Qt5/ANTLR4/SymEngine/Units Stack

Version: 1.0
Date: March 13, 2026
Session: 49 — §2.5 Grok Thread 381a8fe7 Extended Audit
Author: Star-Magic UQFF Research Framework
Source: grok_share_381a8f.txt lines 6400–7000 (S-C Iteration 40, dated 15 Aug 2025)

$$F_U(r,t) = \sum_{i=1}^{4} U_{gi} + U_m + U_A - U_{b_i}, \quad \kappa = 5.0 \times 10^{-4}\,\text{day}^{-1},\ [SSq] = 0.57$$

$$U_{b_i}(r) = \kappa \cdot [SSq] \cdot \frac{GM}{r^2}, \quad \kappa = 5.0 \times 10^{-4}\,\text{day}^{-1},\ [SSq] = 0.57,\ \beta_i = 0.61$$

Abstract

> **Key UQFF calibrated constants:** $\kappa = 5.0\text{e-}4\ \text{day}^{-1}$; [SSq] = 5.7e-1; H_SCm ≈ 9.9e-1; U_UA ≈ 1.0e-4; k_η = 1.0e-113; β_i ≈ 6.0e-1; G = 6.674e-11 $\text{N·m}^2/\text{kg}^2$

This paper documents the complete software architecture of the S-C (Scientific Calculator) Iteration 40, a standalone Qt5-based multi-modal scientific computing environment. The architecture integrates 50+ external libraries spanning: ANTLR4 grammar parsing, SymEngine computer algebra, Eigen linear algebra, TFLite/libtorch machine learning, libsnark zero-knowledge proofs, MPI distributed computing, Qiskit/Cirq quantum simulation, LLVM JIT compilation, Lua scripting, pybind11 Python embedding, VTK 3D visualization, libgit2 version control, pocketsphinx voice recognition, and blockchain transaction support. This represents the most technologically complex component of the Star-Magic ecosystem and constitutes a novel reference implementation for multi-paradigm scientific computing in C++/Qt5.

---

1. Include Stack

1.1 Core Qt5 and Standard Library

```cpp
// Qt5 core
#include <QApplication>
#include <QMainWindow>
#include <QDialog>
#include <QTextEdit>
#include <QWebEngineView>
```

```cpp
#include <QTabWidget>
#include <QCustomPlot>
#include <QVTKOpenGLNativeWidget>

// Networking and collaboration
#include <QNetworkAccessManager>
#include <QWebSocket>
#include <QWebSocketServer>

// C++ standard
#include <memory>
#include <vector>
#include <map>
#include <set>
#include <functional>
#include <thread>
#include <atomic>
```

1.2 Mathematical and Physics Libraries

```cpp
// SymEngine computer algebra
#include <symengine/expression.h>
#include <symengine/symbol.h>
#include <symengine/add.h>
#include <symengine/mul.h>
#include <symengine/pow.h>
#include <symengine/functions.h>
#include <symengine/visitor.h>

// Eigen linear algebra
#include <Eigen/Dense>
#include <Eigen/Sparse>
#include <Eigen/Eigenvalues>

// GNU Scientific Library
#include <gsl/gsl_poly.h>
#include <gsl/gsl_errno.h>
#include <gsl/gsl_matrix.h>
```

1.3 ANTLR4 Parsing

```cpp
#include <antlr4-runtime.h>
#include "MathLexer.h"
#include "MathParser.h"
#include "MathBaseVisitor.h"
#include "MathBaseListener.h"
```

1.4 Machine Learning

```cpp
// TensorFlow Lite
#include <tensorflow/lite/interpreter.h>
#include <tensorflow/lite/kernels/register.h>
#include <tensorflow/lite/model.h>

// PyTorch LibTorch
#include <torch/torch.h>
#include <torch/script.h>
```

1.5 Cryptography and Distributed Computing

```cpp
// Zero-knowledge proofs
#include <libsnark/gadgetlib1/protoboard.hpp>
#include <libsnark/zk_proof_systems/ppzksnark/r1cs_ppzksnark/r1cs_ppzksnark.hpp>

// MPI distributed computing
#include <mpi.h>

// ECDSA signature (via pybind11 ecdsa module)
// Operational transformation
#include <ot/document.h>  // custom OT header
```

1.6 Simulation and 3D

```cpp
// VTK 3D visualization
#include <vtkSmartPointer.h>
#include <vtkSTLWriter.h>
#include <vtkPolyData.h>
#include <QVTKOpenGLNativeWidget.h>

// QuTiP (via pybind11)
#include <pybind11/embed.h>
#include <pybind11/stl.h>

// astropy (via pybind11)
```

1.7 Scripting and Integration

```cpp
// Lua scripting
extern "C" {
    #include <lua.h>
    #include <lualib.h>
    #include <lauxlib.h>
}

// Python embedding
```

```cpp
#include <pybind11/embed.h>

// LLVM JIT
#include <llvm/IR/LLVMContext.h>
#include <llvm/ExecutionEngine/ExecutionEngine.h>
#include <llvm/ExecutionEngine/Orc/LLJIT.h>
```

---

2. Founding Architecture Classes

2.1 SymEngineAllocator

Custom memory allocator for SymEngine expression trees:

```cpp
class SymEngineAllocator {
public:
    void* operator new(size_t size) {
        return ::operator new(size);
    }
    void operator delete(void* ptr) {
        ::operator delete(ptr);
    }
    void* operator new[](size_t size) {
        return ::operator new[](size);
    }
    void operator delete[](void* ptr) {
        ::operator delete[](ptr);
    }
};
```

2.2 Units (7-Dimensional SI System)

```cpp
class Units {
public:
    int mass, length, time, current, temp, amount, luminous;

    Units(int m=0, int l=0, int t=0, int c=0, int T=0, int n=0, int j=0)
        : mass(m), length(l), time(t), current(c), temp(T), amount(n), luminous(j) {}

    Units operator+(const Units& other) const {
        // Units add when multiplying quantities in equation – check same dims
        return *this; // same type
    }

    Units operator-(const Units& other) const {
        return *this; // subtraction requires same units
    }
```

```
    Units operator*(int exp) const {
        return Units(mass*exp, length*exp, time*exp, current*exp,
                     temp*exp, amount*exp, luminous*exp);
    }

    bool operator==(const Units& other) const {
        return mass==other.mass && length==other.length && time==other.time &&
               current==other.current && temp==other.temp &&
               amount==other.amount && luminous==other.luminous;
    }

    std::string toString() const {
        std::string s;
        if (mass!=0) s += "kg^" + std::to_string(mass) + " ";
        if (length!=0) s += "m^" + std::to_string(length) + " ";
        if (time!=0) s += "s^" + std::to_string(time) + " ";
        if (current!=0) s += "A^" + std::to_string(current) + " ";
        if (temp!=0) s += "K^" + std::to_string(temp) + " ";
        return s.empty() ? "dimensionless" : s;
    }
};

// Base unit registry
std::map<std::string, Units> baseUnits = {
    {"kg", Units(1,0,0)},
    {"m",  Units(0,1,0)},
    {"s",  Units(0,0,1)},
    {"A",  Units(0,0,0,1)},
    {"K",  Units(0,0,0,0,1)},
    {"mol",Units(0,0,0,0,0,1)},
    {"cd", Units(0,0,0,0,0,0,1)},
    // Derived units
    {"N",  Units(1,1,-2)},   // Newton
    {"J",  Units(1,2,-2)},   // Joule
    {"W",  Units(1,2,-3)},   // Watt
    {"Pa", Units(1,-1,-2)},  // Pascal
    {"T",  Units(1,0,-2,-1)} // Tesla
};
```

2.3 MathErrorListener

```
class MathErrorListener : public antlr4::BaseErrorListener {
public:
    std::string errorMsg;
    void syntaxError(antlr4::Recognizer* recognizer,
                     antlr4::Token* offendingSymbol,
                     size_t line, size_t charPositionInLine,
                     const std::string& msg,
```

```cpp
                    std::exception_ptr e) override {
        errorMsg = "Syntax error at line " + std::to_string(line) +
                   ", col " + std::to_string(charPositionInLine) +
                   ": " + msg;
    }
    bool hasError() const { return !errorMsg.empty(); }
};
```

2.4 SymEngineVisitor

ANTLR4 visitor that builds SymEngine expression trees with unit propagation:

```cpp
class SymEngineVisitor : public MathBaseVisitor {
    std::map<std::string, SymEngine::RCP<const SymEngine::Basic>> vars;
    std::map<std::string, Units> unitContext;

    antlrcpp::Any visitAdd(MathParser::AddContext* ctx) override {
        auto left  = visit(ctx->expr(0)).as<SymEngine::RCP<const SymEngine::Basic>>();
        auto right = visit(ctx->expr(1)).as<SymEngine::RCP<const SymEngine::Basic>>();
        return SymEngine::add(left, right);
    }

    antlrcpp::Any visitMul(MathParser::MulContext* ctx) override {
        auto left  = visit(ctx->expr(0)).as<SymEngine::RCP<const SymEngine::Basic>>();
        auto right = visit(ctx->expr(1)).as<SymEngine::RCP<const SymEngine::Basic>>();
        return SymEngine::mul(left, right);
    }

    antlrcpp::Any visitPow(MathParser::PowContext* ctx) override {
        auto base = visit(ctx->expr(0)).as<SymEngine::RCP<const SymEngine::Basic>>();
        auto exp  = visit(ctx->expr(1)).as<SymEngine::RCP<const SymEngine::Basic>>();
        return SymEngine::pow(base, exp);
    }

    antlrcpp::Any visitVariable(MathParser::VariableContext* ctx) override {
        std::string name = ctx->IDENTIFIER()->getText();
        if (vars.find(name) == vars.end()) {
            vars[name] = SymEngine::symbol(name);
        }
        return vars[name];
    }

    antlrcpp::Any visitNumber(MathParser::NumberContext* ctx) override {
        double val = std::stod(ctx->NUMBER()->getText());
        return SymEngine::real_double(val);
    }

    antlrcpp::Any visitFunctionDef(MathParser::FunctionDefContext* ctx) override {
```

```
        std::string fname = ctx->IDENTIFIER()->getText();
        auto arg = visit(ctx->expr()).as<SymEngine::RCP<const SymEngine::Basic>>();
        if (fname == "sin") return SymEngine::sin(arg);
        if (fname == "cos") return SymEngine::cos(arg);
        if (fname == "exp") return SymEngine::exp(arg);
        if (fname == "log") return SymEngine::log(arg);
        if (fname == "sqrt") return SymEngine::sqrt(arg);
        return arg; // unknown function ? identity
    }

    antlrcpp::Any visitParametric(MathParser::ParametricContext* ctx) override {
        // Propagate unit context for parametric expressions
        auto expr = visit(ctx->expr()).as<SymEngine::RCP<const SymEngine::Basic>>();
        return expr;
    }
};
```

---

3. ScientificCalculatorDialog Architecture

3.1 Window Configuration

- Size: 800×600 (minimum), resizable
- Window flags: `Qt::FramelessWindowHint | Qt::SubWindow`

- Accepts drops: `setAcceptDrops(true)`
- Gesture support: `grabGesture(Qt::PinchGesture | Qt::SwipeGesture | Qt::PanGesture)`
- Theme: Dark/Light/UQFF toggle

3.2 Primary Widgets

Widget	Type	Purpose
`input`	`QTextEdit`	ANTLR4-highlighted expression input
`output`	`QWebEngineView`	MathJax 3 LaTeX rendering
`scriptEdit`	`QTextEdit`	Lua/Python script editor
`searchBar`	`QLineEdit`	Symbol search (IEF)
`iefIcon`	`QLabel`	Hover-activated IEF search icon
`symbolTabs`	`QTabWidget`	7-category symbol palette
`plot`	`QCustomPlot`	2D function plot
`plotImageLabel`	`QLabel`	ggplot2/Matplotlib image output
`vtkWidget`	`QVTKOpenGLNativeWidget`	3D VTK visualization
`vrScene`	`Qt3DCore::QEntity*`	VR scene graph

3.3 Symbol Palette Categories (7 tabs)

```
Tab 0: Greek   — a ß ? d e ? ? ? ? ? ? µ ? ? p ? s t ? f ? ? ? ? ? G ? ? ? ? T ? ? ? ? ? S (35 chars)
```

```
Tab 1: Operators — + - × ÷ = ? < > = = ± ± ? 8 ˜ = ? ? ? ? ? n ? ? (25 chars)
Tab 2: Functions — ? ? ? ? ? ? ? ? v ? lim sup inf max min (15 chars)
Tab 3: Formulas — common equations (E=mc², F=ma, Schrödinger, Maxwell, etc.)
Tab 4: Physics  — F=ma, E=mc², p=mv, KE=½mv², PE=mgh, F_g=Gm1m2/r², Hooke=kx,
            Ohm=V/IR, P=IV, Q=mc?T, E=hf, de Broglie=h/p (12 physics formulas)
Tab 5: Geometry — circle area, sphere volume, triangle area, Pythagoras, etc.
Tab 6: Motion   — kinematic equations, projectile motion, circular motion
```

3.4 Initialization Sequence

```
1. MPI_Init(&argc, &argv)                         — distributed computing
2. git_libgit2_init()                             — version control
3. ps = ps_init(NULL, NULL)                     — pocketsphinx voice recognition
4. L = luaL_newstate(); luaL_openlibs(L)          — Lua scripting runtime
5. py::scoped_interpreter guard{}                 — Python interpreter
6. sk, vk = py::module::import("ecdsa").generate_keys()  — ECDSA crypto keys
7. MathErrorListener errorListener                 — ANTLR4 error handler
8. MathHighlighter on input                       — syntax highlighting
9. QWebSocketServer on port 8765                  — collaboration server
10. PerlinNoise perlin                             — procedural noise
11.  workspace  =  gsl_poly_complex_workspace_alloc(MAX_POLY_DEGREE)     —
GSL polynomial
12. Load LSTM autocomplete model from "autocomplete.pt"  — torch::jit::load
```

---

4. Technology Integration Map

```
User Input
    ¦
    ?
QTextEdit (input)
    ¦  ANTLR4 MathHighlighter highlights in real-time
    ¦
    ?
solveEquations() --? ANTLR4 parse --? SymEngineVisitor
    ¦                                        ¦
    ¦                               SymEngine::solve()
    ¦                                        ¦
    ¦                               degree > 4? --? GSL poly roots
    ¦                                        ¦
    ¦                               distributed? --? MPI Newton
    ¦                                        ¦
    ¦                               prove it? --? r1cs_ppzksnark ZKP
    ¦
    +--? QCustomPlot auto-plot
    +--? auto-save .csn file
    +--? blockchain transaction
```

```
    +--? broadcastState() --? ECDSA sign --? Snappy compress --? WebSocket

exportResults()
    +-- LaTeX file (.tex)
    +-- PDF via QPrinter
    +-- DOCX via pandoc subprocess
    +-- ODT via QTextDocumentWriter
    +-- MathML via SymEngine::mathml()

simulateMotion()
    +-- Euler integrator (classical)
    +-- QuTiP quantum (pybind11 call)
    +-- astropy solar position (pybind11 call)

forecastSimulation()
   +-- PyTorch LSTM: 1?Dense(50,ReLU)?Dense(1), 10 epochs, forecast 10 steps

importExcel()
    +-- openpyxl cell read + formula eval (via ANTLR4)
    +-- pandas fillna + describe
    +-- scatter/line/bar plot choice

performStats()
    +-- rpy2 ? R: lm, aov, t.test, randomForest, custom, ggplot2 PNG

callGrokAPI()
    +-- biometric gate (QBiometricAuthenticator)
    +-- POST to api.x.ai/v1/chat/completions
    +-- model: grok-beta
```

---

5. Conclusion

S-C Iteration 40 constitutes the most architecturally comprehensive component of the Star-Magic ecosystem. Its 50+ library integration stack is unprecedented in open-source Qt5 applications and creates a unified platform for: symbolic mathematics (SymEngine+ANTLR4), numerical computation (Eigen+GSL+MPI), machine learning (TFLite+libtorch), quantum simulation (Qiskit via pybind11), cryptographic verification (libsnark ZKP + ECDSA), collaborative editing (WebSocket+OT), and scientific visualization (VTK+QCustomPlot). Three subsequent papers (PAPER_190–192) document specific functional modules of this architecture.

References

- Source: grok_share_381a8f.txt lines 6400–7000

- Related: PAPER_190 (Symbolic Integration), PAPER_191 (Multi-Modal Features), PAPER_192 (Collaborative Math)
- CP1 Class: `CoAnQiScientificCalculatorArchitectureCalculator`